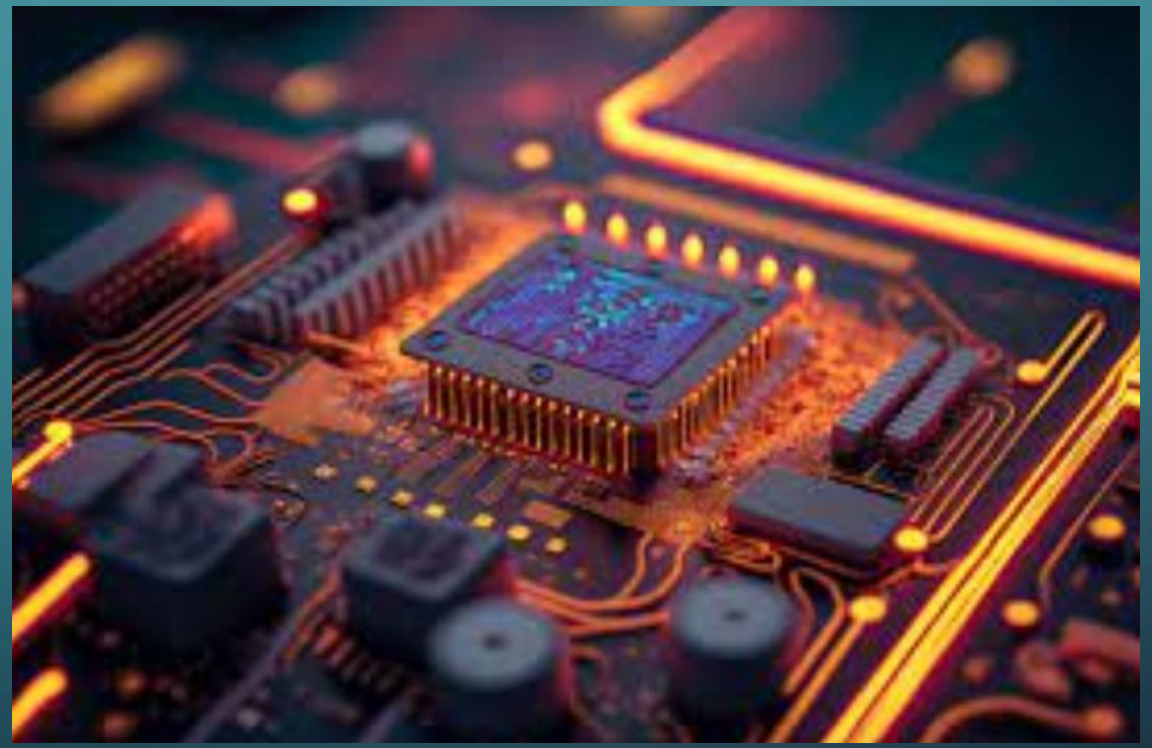
UTN
Facultad Regional Córdoba

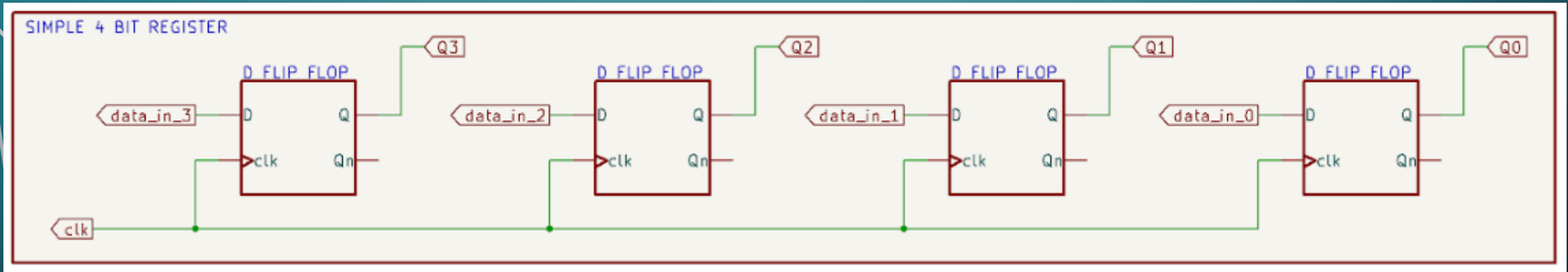
REGISTEROS EN VERILOG



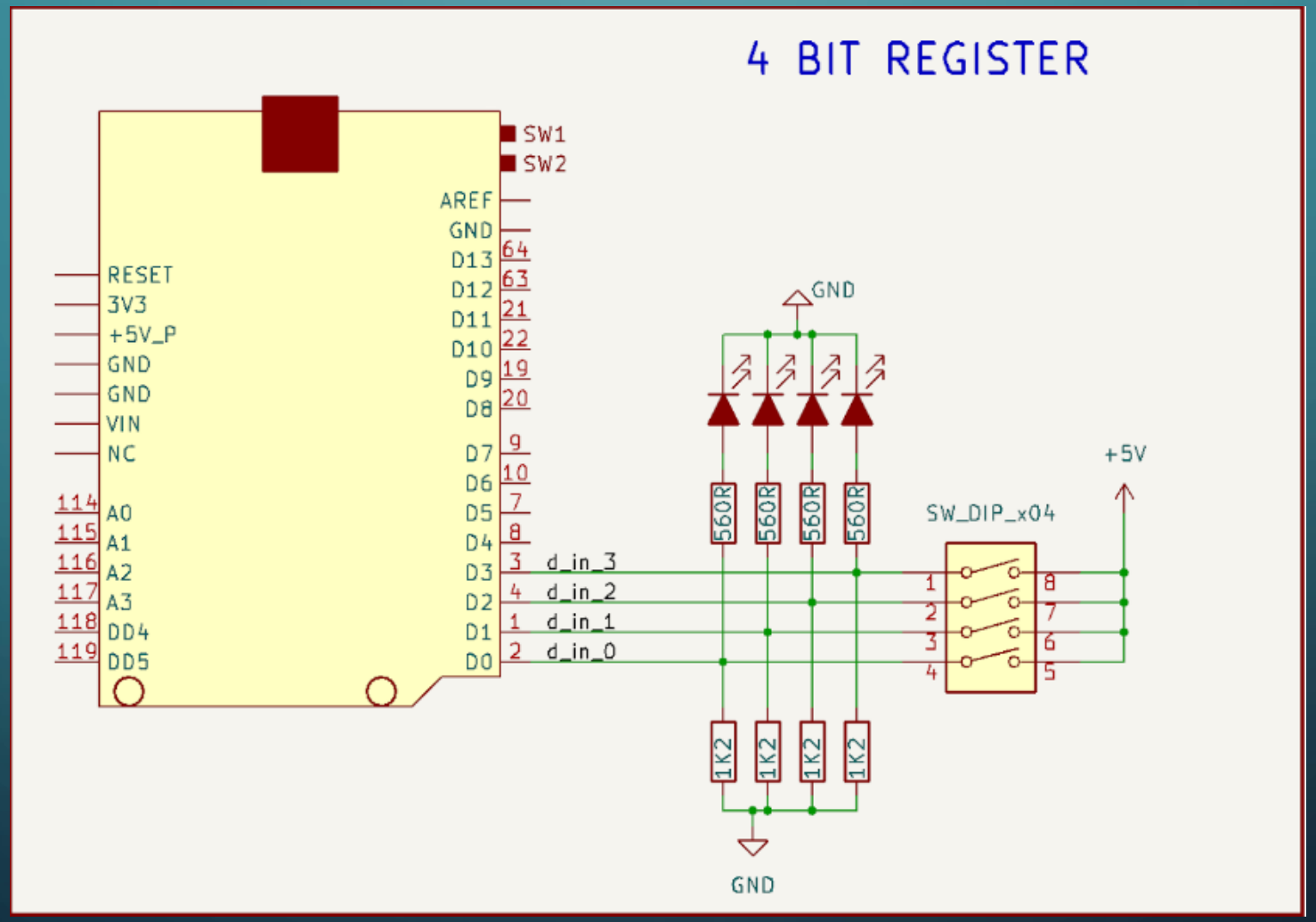
Ing. Marcelo Casasnovas Año 2026

Registro sencillo de 4 bits

Los registros se utilizan para almacenar datos, por ejemplo el estado actual de una Máquina de Estado Finito (FSM) o como puerto de E/S cuando se utilizan como interfaz de comunicaciones serie tal como SPI, I2C



El dato se introduce de forma paralela en los pines **D0** a **D3** mediante un dip switch. Unos ledes nos ayudan a visualizar el dato seleccionado. La salida **Q** la vemos reflejada en los ledes 4 a 7 donde el **LED7** es el menos significativo.



```
// 4 bit simple register

module simple_4_bit_register ( input clk, input [3:0] data_in,
                               output reg [3:0] Q );

reg [3:0] Q_next;

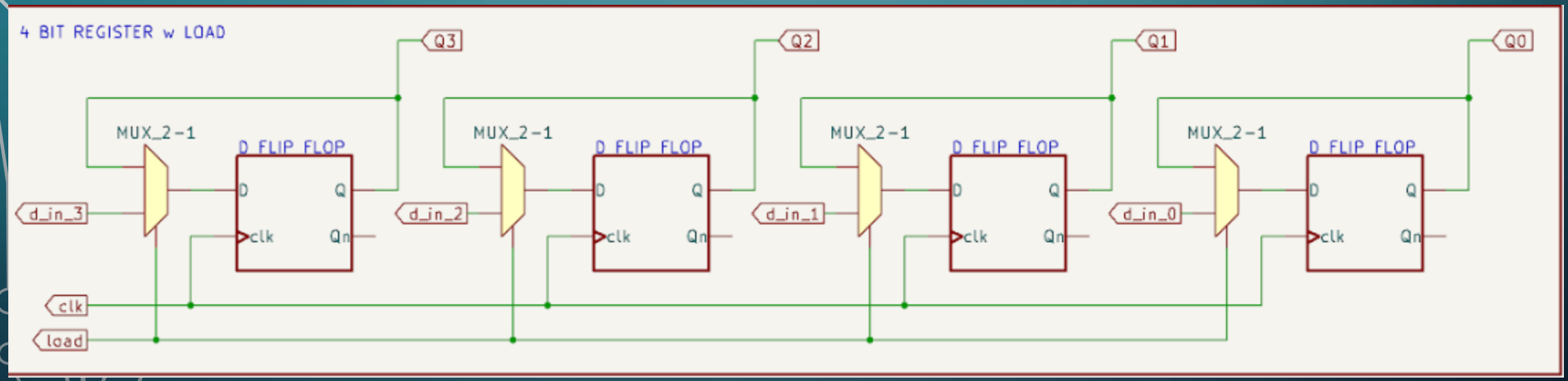
always @ (posedge clk)
begin
  Q <= Q_next;
end

always @ (data_in)
begin
  Q_next = data_in;
end

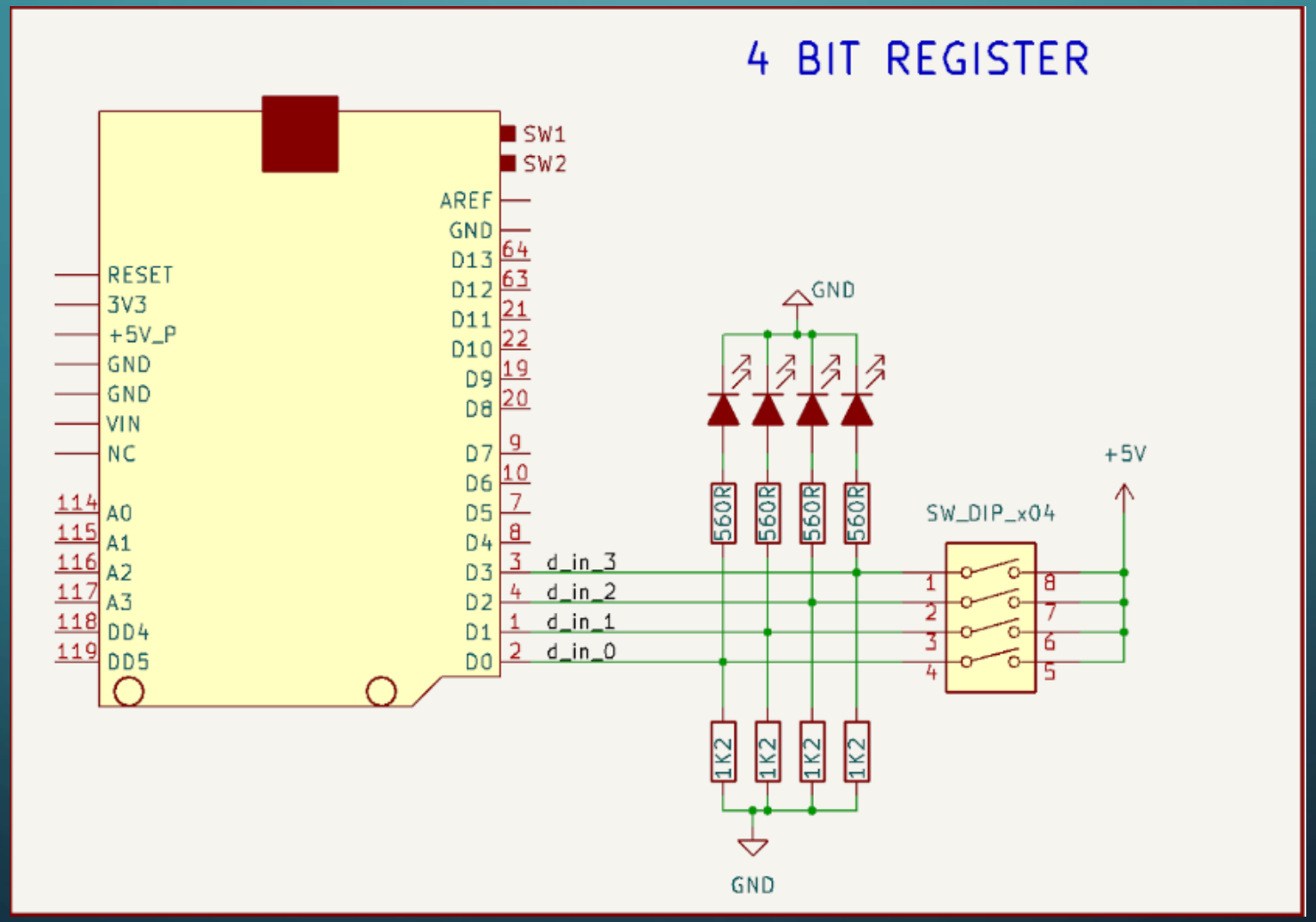
endmodule
```

Registro de 4 bits con carga paralela

En el siguiente ejemplo vamos a introducir una mejora al registro anterior. Añadiremos un multiplexor 2-1 en cada entrada **`D`** de los flip-flop individuales. El multiplexor nos permite seleccionar el dato que introducimos en el flip-flop, entre **`d_in`** y **`Q`**, de manera que en reposo el registro mantiene el dato actual y al pulsar **`load`** cargamos un nuevo dato.



Seleccionamos el dato con un dip switch y lo cargamos con el pulsador **sw1**



```
module register_4_bit_w_load ( input clk, load, input [3:0] d_in,
                               output reg [3:0] Q );

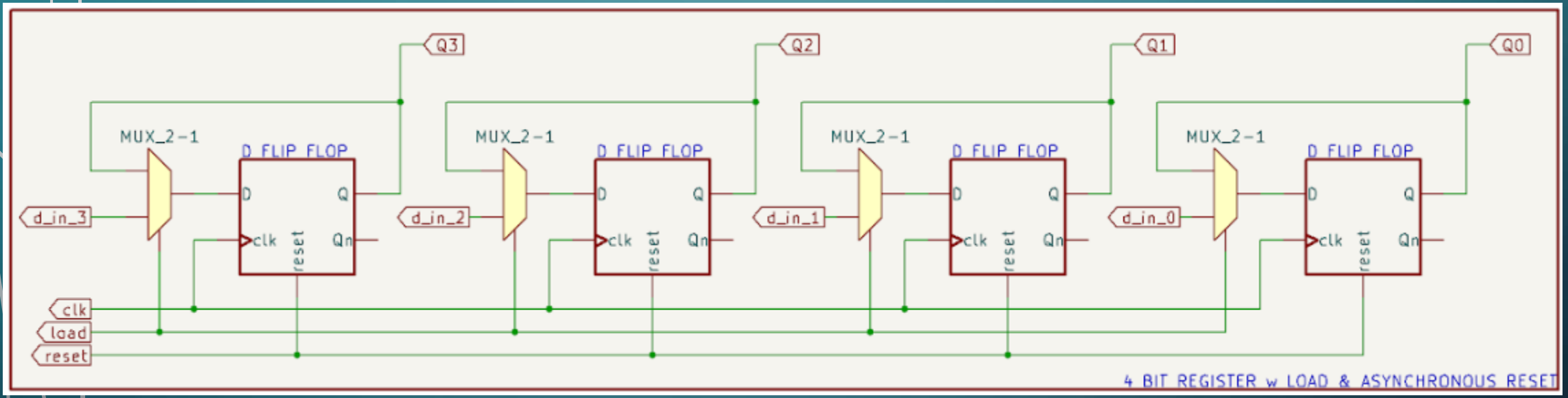
reg [3:0] Q_next;

always @ (posedge clk)
begin
  Q <= Q_next;
end

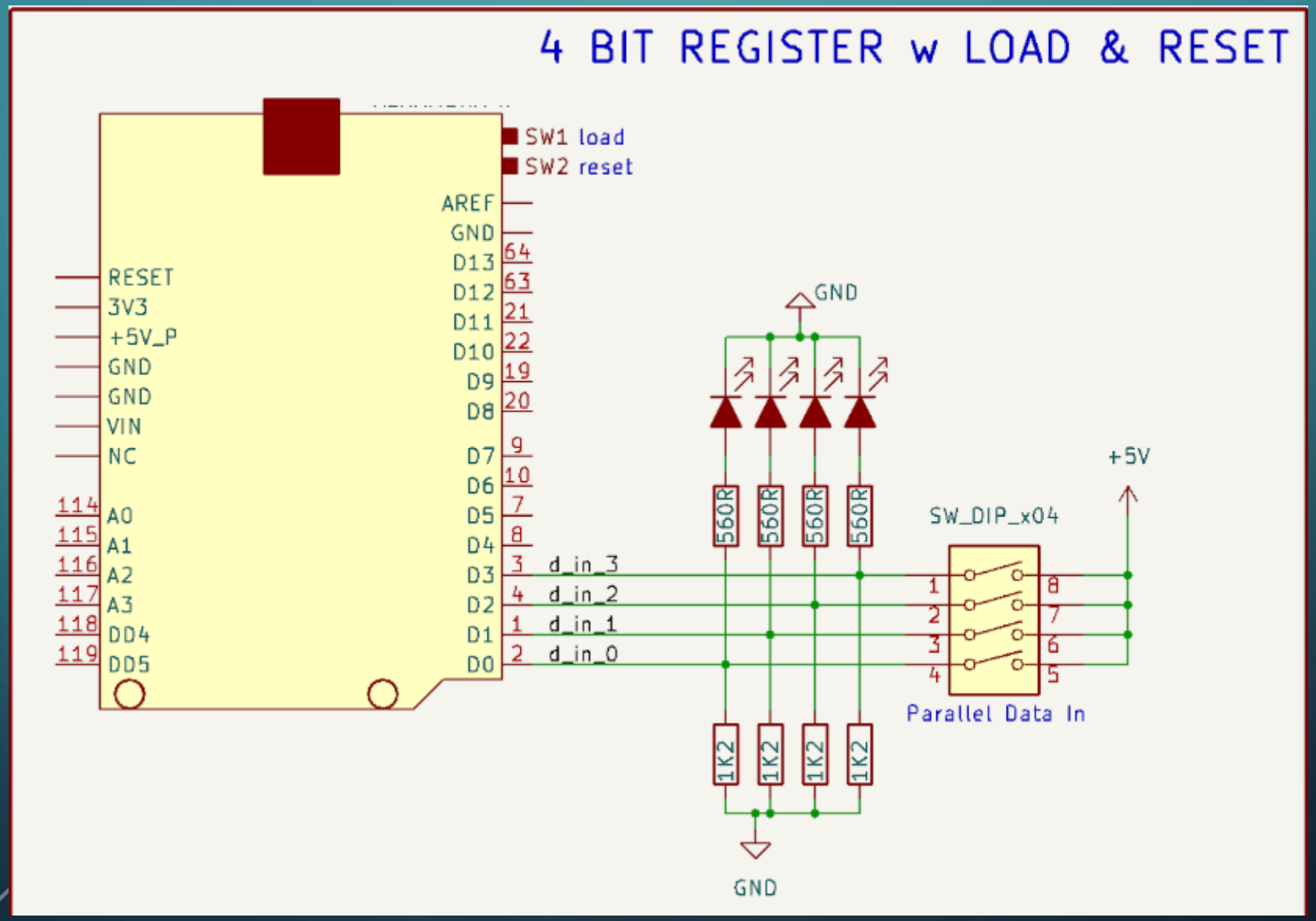
always @ (d_in, load)
begin
  if (load)
    Q_next = d_in;
  else
    Q_next = Q;
end
```

Registro de 4 bit con carga y reset asíncrono

Con el mismo circuito que en el ejemplo anterior, vamos a añadir un pulsador de reset asíncrono para poner las salidas a cero sin importar la señal de reloj.



El pulsador de reset lo implementamos en el **sw2**



```
module register ( input clk, load, reset, input [3:0] d_in,
                  output reg [3:0] Q );

reg [3:0] Q_next;

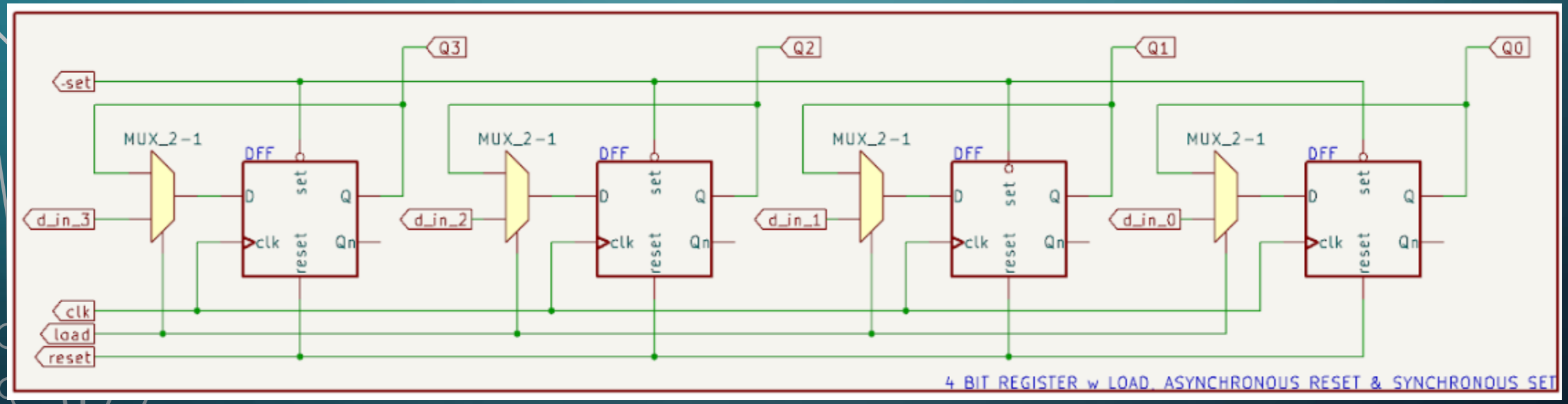
always @ (posedge clk or posedge reset)
begin
  if (reset)
    Q <= 4'b0000;
  else
    Q <= Q_next;
end

always @ (d_in, load)
begin
  if (load)
    Q_next = d_in;
  else
    Q_next = Q;
end

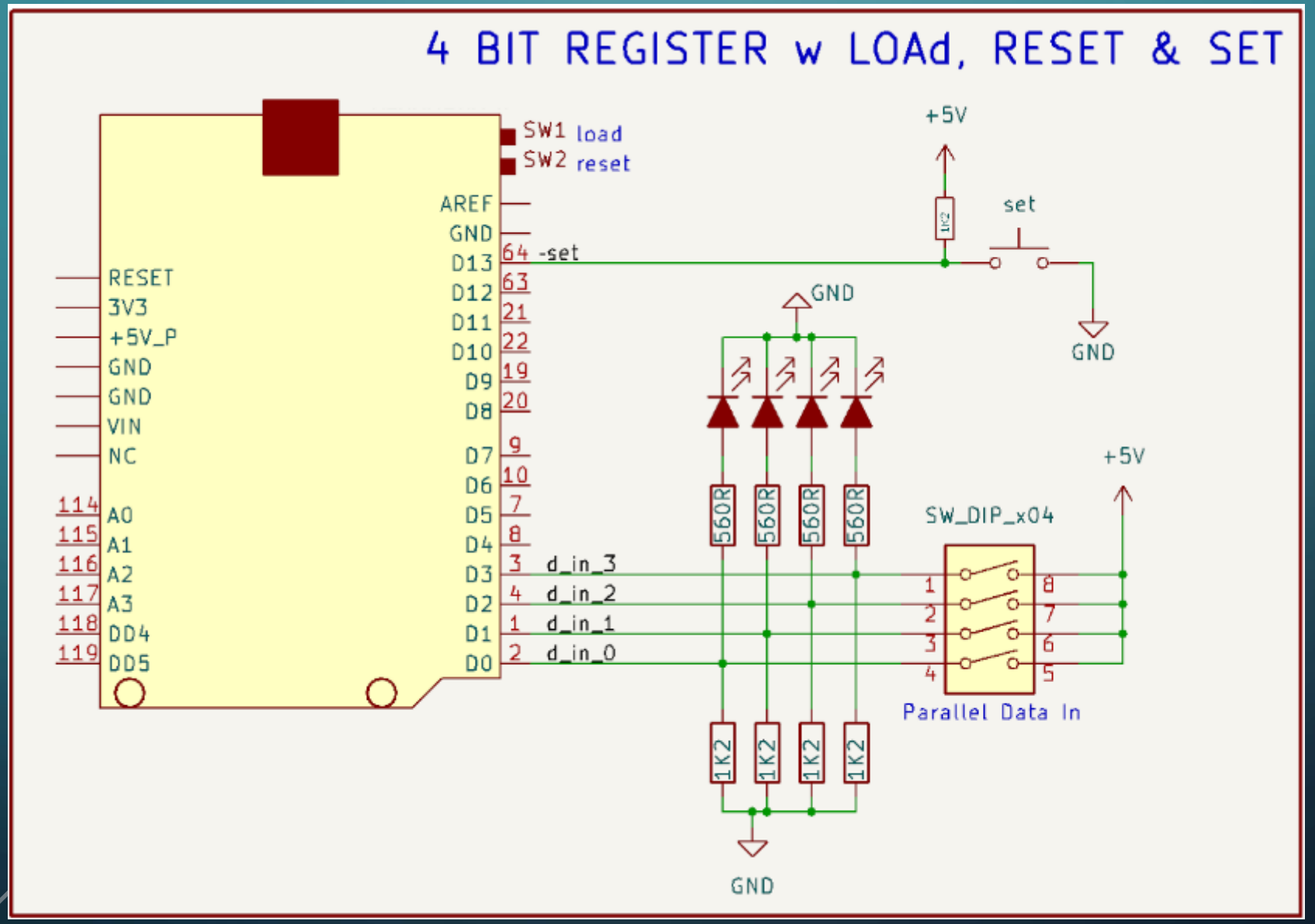
endmodule
```

Registro de 4 bit con carga, reset asíncrono y set síncrono

Para terminar con los registros, en este último ejemplo vamos a modelar un registro con **reset** asíncrono y **set** síncrono activo a nivel bajo. El pulsador **set**, pone todas las salidas a 1 si está pulsado durante el flanco de subida del reloj.



La señal de **set** la introducimos en pin **D13** con un pulsador externo. Al ser una señal activa a nivel bajo, conectamos el pulsador a GND y en la entrada **D13** conectamos una resistencia pull-up de 1K2.



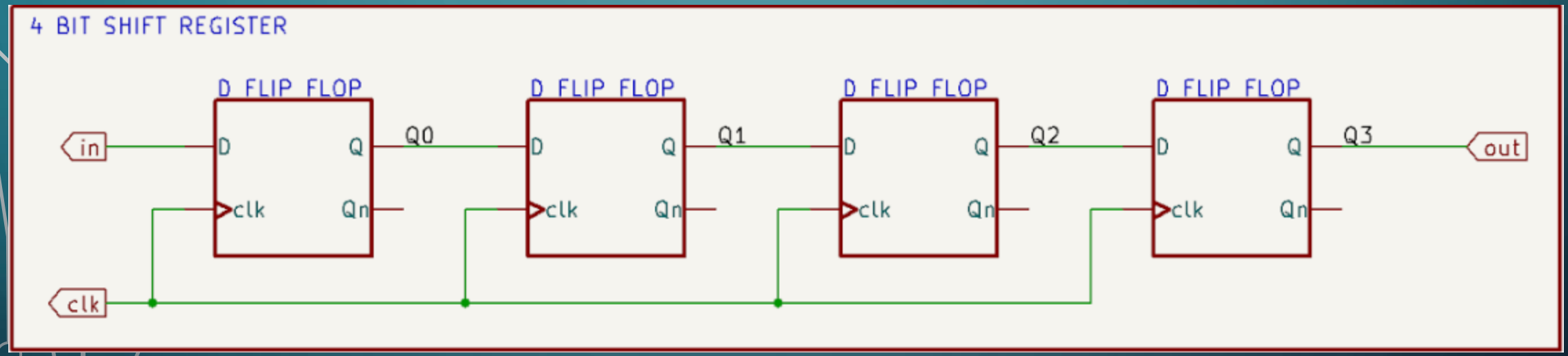
```
module register_w_load_reset_set ( input clk, load, reset, set,
                                   input [3:0] d_in,
                                   output reg [3:0] Q );
reg [3:0] Q_next;

always @ (posedge clk or posedge reset)
begin
  if (reset)
    Q <= 4'b0000;
  else if (!set)
    Q <= 4'b1111;
  else
    Q <= Q_next;
end

always @ (d_in, load)
begin
  if (load)
    Q_next = d_in;
  else
    Q_next = Q;
end
endmodule
```

Registros de desplazamiento

Los registros de desplazamiento o *Shift Registers*, se utilizan ampliamente en protocolos de comunicación para transmitir datos en serie, utilizando un sola línea de comunicación y disminuyendo así el número de cables necesarios.
También se utilizan para hacer operaciones matemáticas, tales como multiplicar y dividir.



Registro de desplazamiento de 4 bits

Para empezar vamos a modelar un registro de desplazamiento sencillo, de 4 bits, con desplazamiento hacia la derecha.
Con el pulsador **sw1** introducimos los datos en serie. El dato aparece en el bit más significativo **Q[3]** que se corresponde con el **LED4** y se va desplazando hacia la derecha por los ledes **LED5**, **LED6** y **LED7** de la FPGA.
Para ver mejor el funcionamiento del registro, se deberá utilizar la señal de 1 Hz que será la que aplicaremos en la entrada de los flip-flop.

Confidential C

```
module shift_register (input clk,
                       input  serial_in,
                       output reg clk2,
                       output reg [3:0] Q
                       );

reg [31:0] counter = 0;

// slow clk
always @ (posedge clk)
  begin
    if (counter < 6000000) counter <= counter + 1;
    else
      begin
        clk2 <= !clk2;
        counter <= 0;
      end
  end

always @(posedge clk2)
begin
    Q[3] <= serial_in;
    Q[2] <= Q[3];
    Q[1] <= Q[2];
    Q[0] <= Q[1];
end

endmodule
```

Confidential C

Para describir un registro con desplazamiento a la izquierda, sólo hay que cambiar el segundo bloque **always** por el que se muestra a continuación:

```
always @(posedge clk2)
begin
    Q[3] <= Q[2];
    Q[2] <= Q[1];
    Q[1] <= Q[0];
    Q[0] <= serial_in;
end
```

También se puede escribir de la siguiente forma:

```
always @ (posedge clk2)
begin
// Right shift
  Q <= {serial_in, Q[3: 1]};

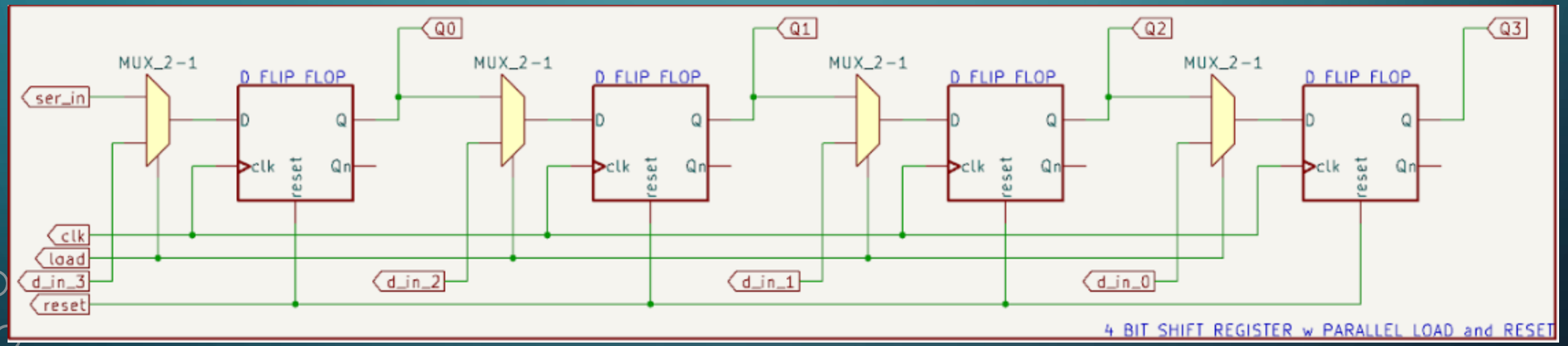
// Left shift
// Q <= {Q[2:0], serial_in};
end
```

El ejemplo que acabamos de ver tiene el problema de que está siempre funcionando; el registro está continuamente desplazando los bits con cada flanco de la señal de reloj.

Confidential C

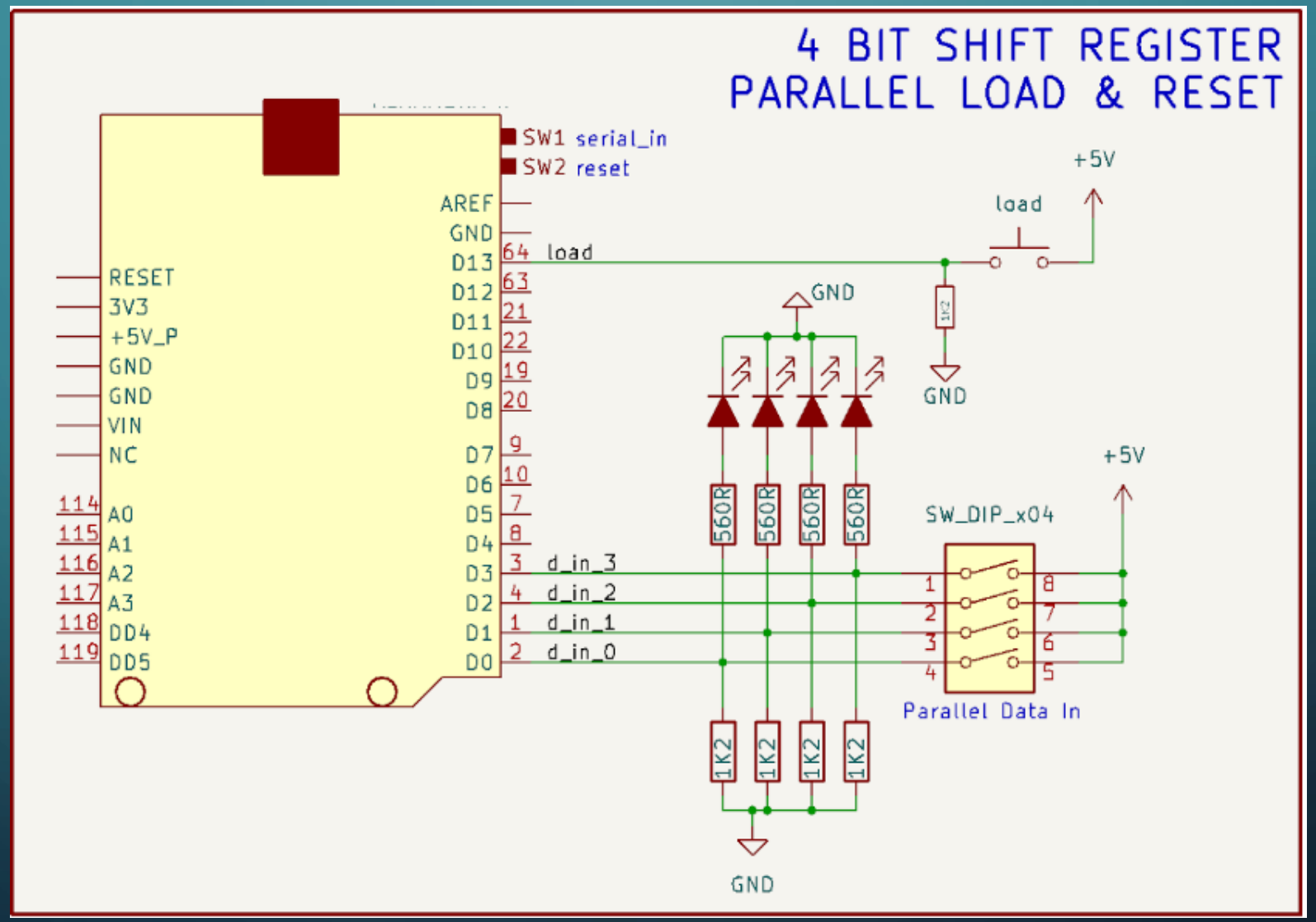
Registro de desplazamiento de 4 bits con carga paralela y reset.

Vamos a agregar un pulsador de **`load`** para cargar un dato de forma paralela en los flip-flop individuales.
Mantenemos también la entrada **`serial_in`** para introducir datos en serie.
Con un pulsador de **`reset`** asíncrono hacemos la salida **`0`** sin importar la señal de reloj.



Confidential C

Para introducir los datos en serie utilizamos el **sw1**. Con el **sw2** reseteamos la salida.
La opción de **load** la implementamos con un pulsador externo conectado en el pin **D13**.
El dato paralelo se selecciona con un dip switch y lo introducimos en los pines **D0** a **D3**.



Confidential C

```
module shift_register (input clk,
                       input  serial_in,
                       input [3:0] data_in,
                       input load,
                       input reset,
                       output reg clk2,
                       output reg [3:0] Q
                       );

reg [31:0] counter = 0;

// custom clk
always @ ( posedge clk)
  begin
    if (counter < 6000000) counter <= counter + 1;
    else
      begin
        clk2 <= !clk2;
        counter <= 0;
      end
  end
```

Confidential C

```
always @ (posedge clk2 or posedge reset)
begin
  if (reset)
    Q <= 8'b0000_0000;
  else if (load)
    Q <= data_in;
  else
    // Right shift
    Q <= {serial_in, Q[3: 1]};

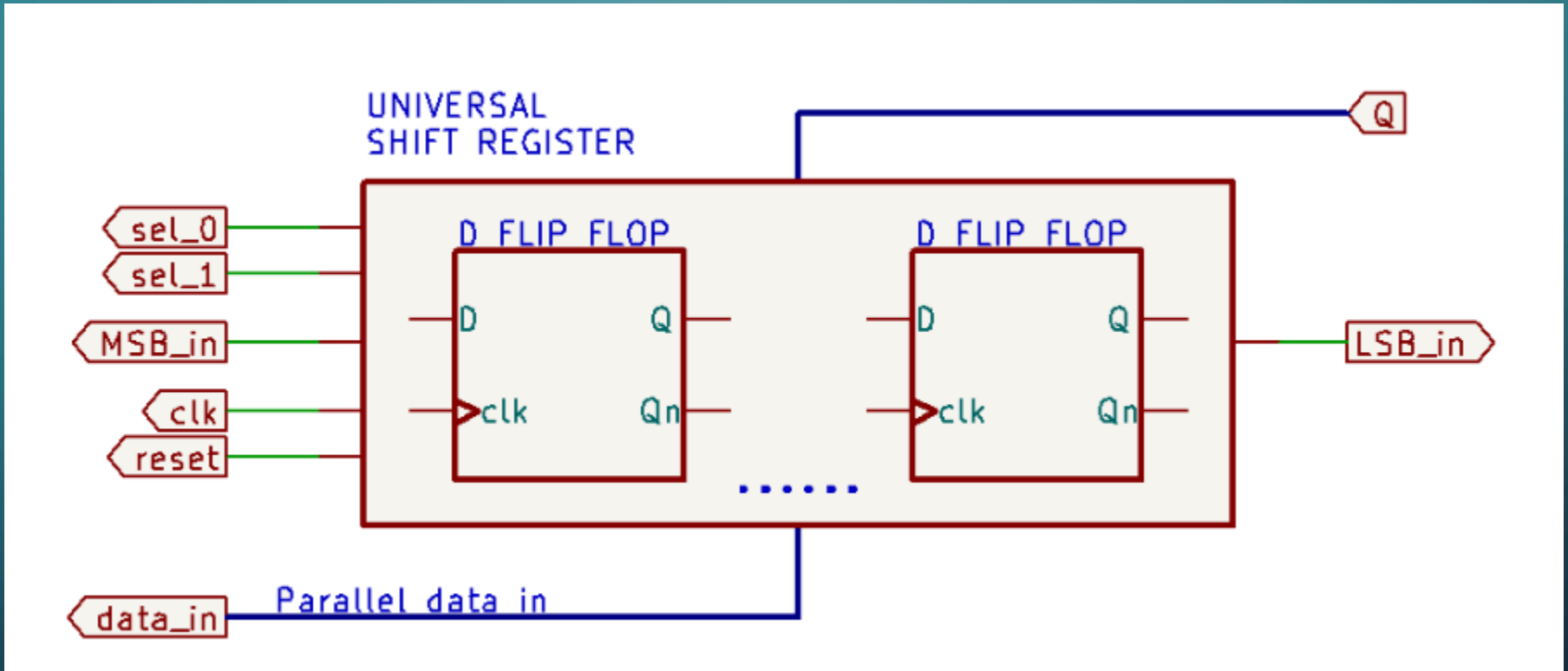
    // Left shift
    // Q <= {Q[2:0], serial_in};
end

endmodule
```

Confidential C

Registro de desplazamiento universal

Un registro de desplazamiento universal, agrupa todas las funcionalidades típicas de los registros de desplazamiento, tales como: desplazamiento a derecha e izquierda, carga paralela, reset, set y entrada serie por el bit más -y menos- significativo.



Confidential C

Para seleccionar el modo de funcionamiento empleamos dos entradas **`sel_0`** y **`sel_1`** de acuerdo con la siguiente tabla.

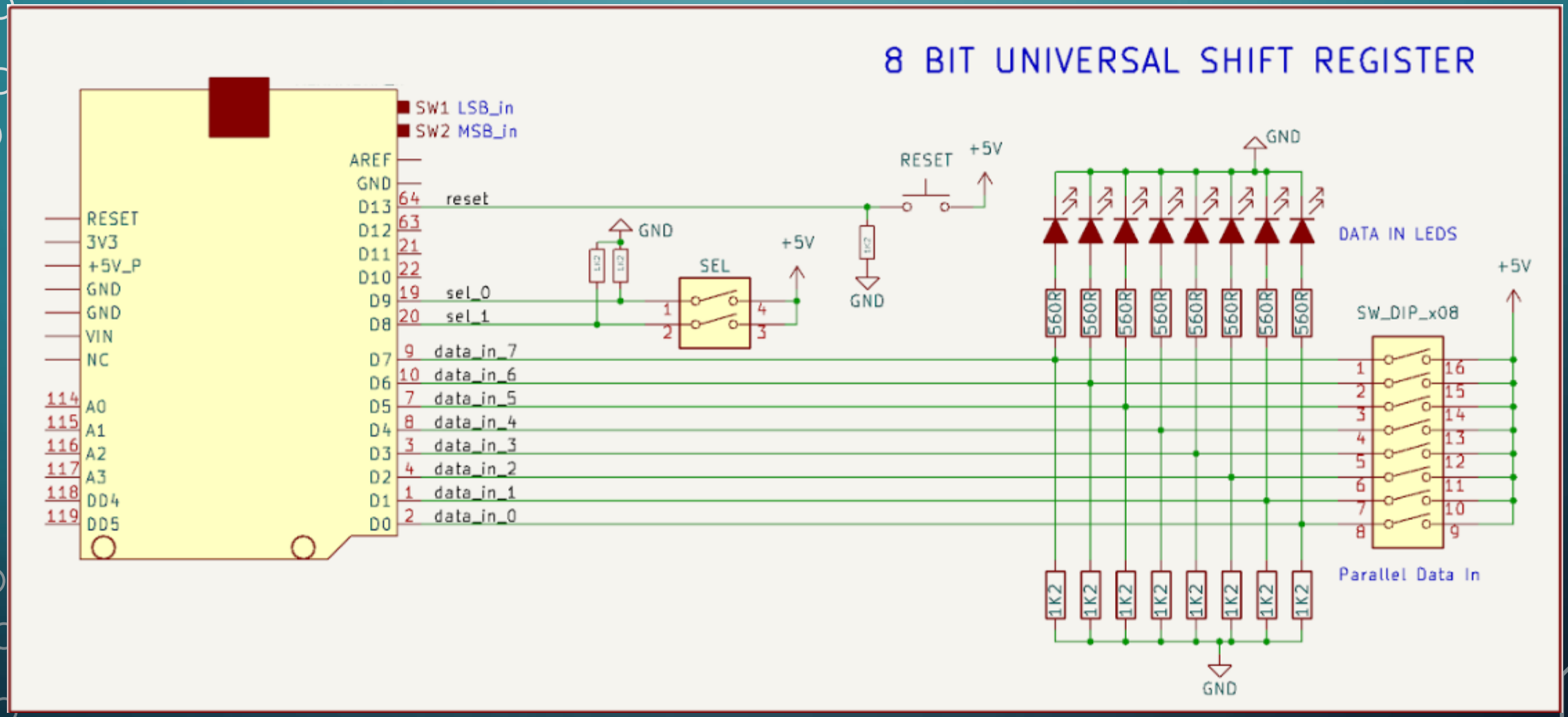
sel_0	sel_1	Operation Mode
0	0	no change
0	1	shift right
1	0	shift left
1	1	parallel load

Confidential C

Esquema

Se va a modelar un registro universal de 8 bits.
Con los pulsadores **`sw1`** y **`sw2`** implementamos las entradas serie **`LSB_in`** y **`MSB_in`**.
La función de **`reset`** la realizamos con un pulsador conectado en el pin **`D13`**.
Con un dip switch doble, seleccionamos el modo de funcionamiento.
La entrada de datos la realizamos en los pines **`D0`** a **`D7`**.
La salida la mostraremos en los ledes de la FPGA.

Confidential C



Confidential C

```
module univ_shift_reg (
        input clk, reset,
        input  MSB_in, LSB_in,
        input [7:0] data_in,
        input [1:0] sel,
        output [7:0] Q );

reg [7:0] Q_reg, Q_next;
reg [31:0] counter = 0;
reg clk2;

// slow clk
always @ ( posedge clk)
begin
  if (counter < 6000000) counter <= counter + 1;
  else
    begin
      clk2 <= !clk2;
      counter <= 0;
    end
end
```

Confidential C

```
always @ (posedge clk2, posedge reset)
begin
  if (reset)
      Q_reg <= 0;
  else
      Q_reg <= Q_next;
end

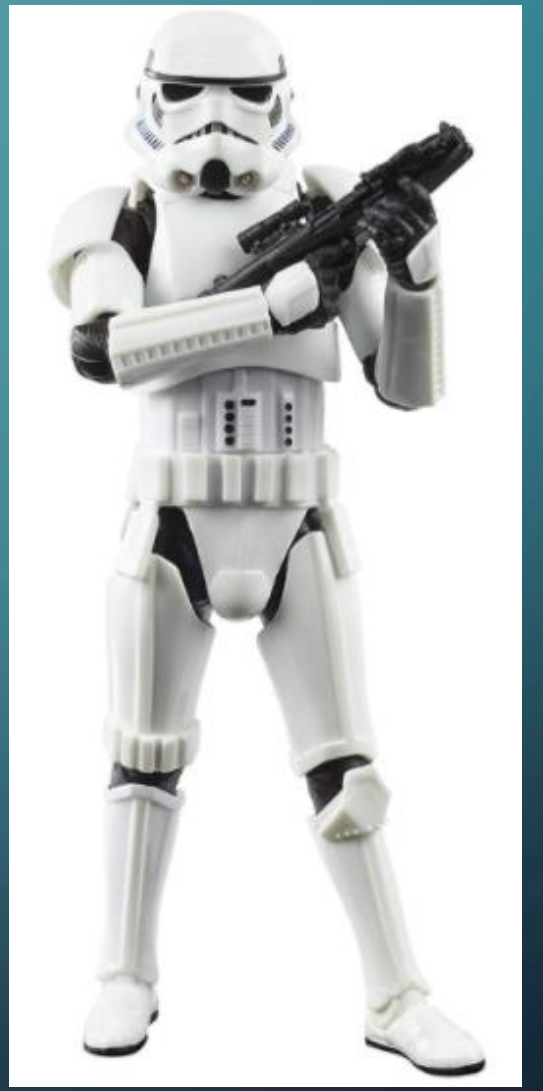
// Next state logic
always @ (Q_reg, MSB_in, LSB_in, data_in, sel)
  begin
    case (sel)
        2'b00: Q_next = Q_reg;                // No change
        2'b01: Q_next = {MSB_in, Q_reg[7:1]}; // Shift right
        2'b10: Q_next = {Q_reg[6:0], LSB_in}; // Shift left
        2'b11: Q_next = data_in;              // Parallel load
        default: Q_next = Q_reg;
    endcase
end

// output logic
assign Q = Q_reg;

endmodule
```

Confidential C

PREGUNTAS ?????



Confidential C